EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 3 · Teoría y práctica en directo

Malasaña · Goya · Eixample



APERTURA

Hoy, última sesión: terminar lo empezado y verlo funcionar entero

I

Terminar el RAG

Trocear, buscar, versiones y citas

MIN 8–45

Prácticas 5 y 6 (las de la S2)

II

Leer lo que no es texto

Un ticket y los PDF de junio

MIN 45–85

Prácticas 1 y 2

III

Del script al asistente

API, Open WebUI y LangGraph

MIN 95–115

Práctica 3

IV

El asistente, en directo

Preguntas difíciles, de punta a punta

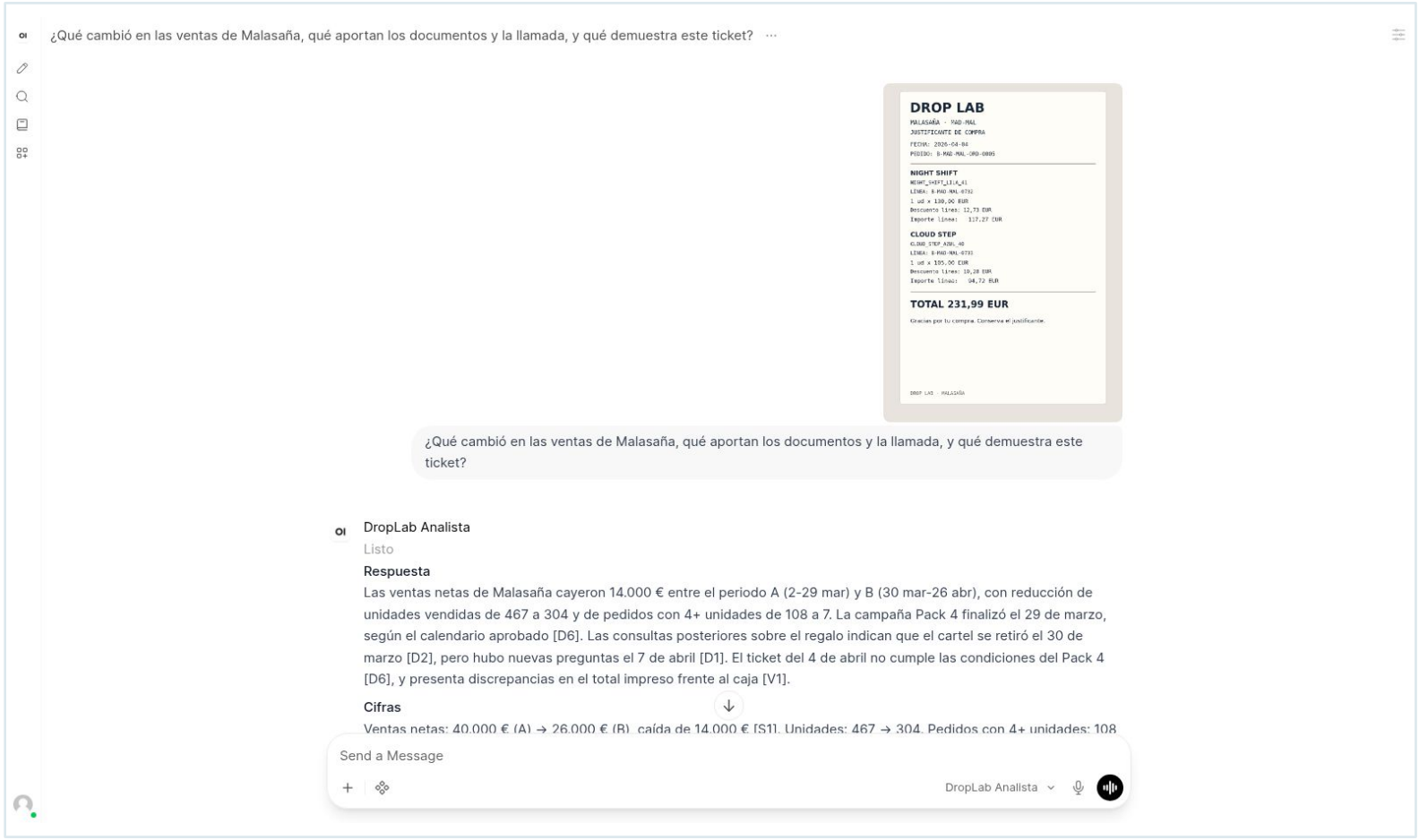
MIN 115–150

En directo, juntos

Descanso: 85–95 · Cierre y entrega final: 145–150

Un hilo: **un modelo lee y redacta; el código calcula, comprueba y dice lo que no se sabe.**

Lo que vamos a entender hoy: el Asistente DropLab, en el navegador



Escribes una pregunta
y adjuntas un ticket

Una API la recibe
FastAPI, en el Jetson

Un flujo decide
imagen · cifras · documentos

El código comprueba
caja, versiones, lecturas

El modelo redacta
citando [S#] [D#] [V#]

Test de la sesión 2: la clave de las diez primeras

#	La idea	Resp.
1	Capa semántica: vistas en código que ya resuelven periodos, abonos y tiendas.	B
2	El abono va al periodo de su fecha de reconocimiento, no al de la venta.	C
3	El verificador no ve «307 líneas» si 307 aparece en un resultado (eran unidades).	D
4	Batería para diseñar; validación y prueba final para medir sin hacerse trampas.	A
5	MoE: ~3 B parámetros activos por token, pero los 30 B en memoria.	B
6	Whisper: log-mel, codificador y decodificador que escribe tokens como un LLM.	C
7	Con poca señal, el decodificador completa algo plausible: alucinación.	A
8	La probabilidad por palabra prioriza qué escuchar; no certifica.	B
9	Quién habla: un canal por interlocutor, no reconocimiento de voz.	D
10	Ajuste fino: revisar etiquetas, separar por llamada y medir el WER antes y después.	D

Las preguntas 11–15 (RAG), al terminar el bloque I.

EBIS Business Techschool

BLOQUE I · TERMINAR EL RAG

Terminar el RAG

Preparar, buscar y responder con fuentes: lo que quedó pendiente de la sesión 2.

Pregunta guía
¿Qué dicen los documentos y la llamada, y dónde lo dicen?

EN ESTE BLOQUE

1. **Preparar las fuentes**
   Trocear por su estructura y poner a cada fragmento su ficha.
2. **Buscar bien**
   Embeddings, palabras clave y búsqueda híbrida.
3. **La trampa de las versiones**
   Lo más parecido a la pregunta es… el borrador.
4. **Responder con fuentes**
   Citas, un código que revisa al modelo… y lo que no ve.

Un RAG en cuatro pasos, todo en el Jetson

1

Trocear y etiquetar

10 fuentes → 51 fragmentos con su ficha

`10_corpus.py`

2

Índice vectorial

cada fragmento → 1.024 números (bge-m3), en DuckDB

`11_indice.py`

3

Buscar

la pregunta → los 5 fragmentos más relevantes

`12_buscar.py`

4

Responder y verificar

solo con esos fragmentos, citando [F#]

`13_preguntar_documentos.py`

LAS FUENTES (Y NUNCA LA CARPETA DEL PROFESOR)

6 documentos Markdown | **3 PDF (incluidos expediente y manual)** | **La llamada (Whisper turbo, solo voz)**

Embeddings y modelo en Ollama, índice en un archivo DuckDB: nada sale del equipo.

EBIS Business Techschool

PRÁCTICA 5 · BLOQUE I

Preparar y buscar

De diez fuentes a un índice que encuentra el fragmento justo, con su ficha.

▸ Ahora, en la terminal

AL TERMINAR

Las slides siguientes explican el troceado, la ficha, los embeddings y la búsqueda.

PASOS

1 Trocear y etiquetar las fuentes

```
python practica/sesion_2/10_corpus.py
```

→ 10 fuentes → 51 fragmentos: un ■ por fragmento, del color de su estado.

2 Ver cómo se corta un documento

```
python practica/sesion_2/10_corpus.py --trocear calendario
```

→ Antes y después: qué va a la ficha, qué se quita y dónde se corta.

3 Crear el índice vectorial

```
python practica/sesion_2/11_indice.py
```

→ 51 vectores de 1.024 números (bge-m3) en DuckDB, en unos 15 s.

4 Buscar

```
python practica/sesion_2/12_buscar.py "¿Hasta qué día estuvo vigente el Pack 4?" --explicar
```

→ El primero es… el borrador. Y el buscador añade el vigente.

Así se trocea un documento: por secciones, con su ficha

calendario_comercial_aprobado.md

✂ cabecera → la ficha (no es un fragmento)

```
# DropLab · Calendario comercial aprobado                    → título
Documento CAM-2026-03-FINAL · Versión 2 · Estado: APROBADO   → código, estado
Aprobación: 01/03/2026 · Responsable: dirección comercial    → fecha, responsable
```

✂ sección «Campañas» → fragmento #2

```
## Campañas
| Tienda | Campaña | Inicio | Último día incluido |...
| MAD-MAL | Pack 4 | 02/03/2026 | 29/03/2026 | Tote bag...
| MAD-GOY | Semana Running | 09/03/2026 | 15/03/2026...
| BCN-EIX | Sin campaña local | 02/03/2026 | 26/04/2026...
```

✂ sección «Pack 4: ejecución» → fragmento #3

```
## Pack 4: ejecución
Incluye zapatillas, camisetas y sudaderas. Cada par...
Pack 4 no modifica los precios de lista ni añade...
Este calendario sustituye a CAM-2026-03-DRAFT...   → DRAFT: sustituido
Operaciones no incluyó en la exportación el registro...
```

#2 calendario_comercial_aprobado.md · Campañas

CAM-2026-03-FINAL | aprobado | 01/03/2026 | dirección comercial

TEXTO QUE SE INDEXA

Calendario comercial aprobado (CAM-2026-03-FINAL, aprobado, 01/03/2026) · Campañas

```
| MAD-MAL | Pack 4 | 02/03/2026 | 29/03/2026 | Tote bag...
```

#3 calendario_comercial_aprobado.md · Pack 4: ejecución

CAM-2026-03-FINAL | aprobado | 01/03/2026 | dirección comercial

TEXTO QUE SE INDEXA

Calendario comercial aprobado (CAM-2026-03-FINAL, aprobado, 01/03/2026) · Pack 4: ejecución

```
Incluye zapatillas, camisetas y sudaderas. Cada par...
```

La cabecera **azul** (de la ficha) se pega delante del texto: el fragmento se entiende aunque llegue solo.

PRÁCTICA 5 · 10_corpus.py --trocear calendario

Cada fragmento lleva su ficha: lo exige el propio manual

10_corpus.py --ver 36

```
"cita":     "propuesta_pack4_borrador.md · Propuesta"
"titulo":   "Propuesta de campaña Pack 4"
"codigo":   "CAM-2026-03-DRAFT"
"fecha":    "27/02/2026"
"responsable": "equipo comercial"
"estado_documento": "borrador"
"estado":   "sustituido"
"sustituido_por": "CAM-2026-03-FINAL"

"texto": "Tienda: MAD-MAL. Inicio propuesto: 02/03/2026.
          Último día propuesto: 31/03/2026. …"
```

MANUAL DE DATOS · PÁG. 7

«Todo fragmento utilizado en un informe debe conservar el documento de origen, sección o página, fecha, responsable y estado documental.»

«Sustituido»: regla literal en Python

El código busca «sustituye a CAM-2026-03-DRAFT», conserva la cita donde lo encontró y actualiza el estado de las copias del borrador. Es una inferencia por regla para este caso, no una validación general de versiones; el embedding no la decide.

PRÁCTICA EXTRA si sobra tiempo · 10_corpus.py --ver 36

Embeddings: el significado convertido en números



SIMILITUD COSENO · BGE-M3

«avería del vehículo»
↔ «el camión se estropeó»

0,67

«tote bag»
↔ «bolsa de regalo»

0,67

«tote bag»
↔ «HIST-JOIN-001»

0,39

Sin una palabra en común, y cerca.

El problema
Borrador y aprobado dicen casi lo mismo: quedan juntos.

PRÁCTICA EXTRA si sobra tiempo · 11_indice.py --comparar

Tres formas de buscar

Semántica

por significado (embeddings)

✓ **Entiende «vigencia» ≈ «último día»**

✗ **Puede perder códigos raros**

Palabras (BM25)

por palabras clave, como un buscador clásico

✓ **Clava «HIST-JOIN-001»**

✗ **No entiende sinónimos**

Híbrida

las dos, unidas por su posición (RRF)

✓ **Lo mejor de las dos**

✗ **Un poco más lenta**

Con «¿hasta cuándo duró el Pack 4?» falla la de palabras; con «HIST-JOIN-001», la semántica duda.

PRÁCTICA EXTRA si sobra tiempo · 12_buscar.py "HIST-JOIN-001" --modo palabras

La trampa de las versiones: lo más parecido es el borrador

«¿HASTA QUÉ DÍA ESTUVO VIGENTE EL PACK 4?» · BÚSQUEDA HÍBRIDA

1	expediente · pág. 7 · Propuesta inicial	sustituido · 31/03
2	propuesta_pack4_borrador.md · Pendientes	sustituido
3	expediente · pág. 2 · Calendario aprobado	aprobado · 29/03
4	propuesta_pack4_borrador.md · Propuesta	sustituido · 31/03
5	calendario_comercial_aprobado.md · Pack 4	aprobado
+	calendario_comercial_aprobado.md · Campañas	añadido: el vigente

TRES DEFENSAS

Estado en la ficha
«sustituido por CAM-2026-03-FINAL»

El código añade el vigente
si sale un sustituido, trae el que lo sustituye

Una regla para el modelo
manda el aprobado; el borrador, solo como antecedente

La similitud no sabe **qué documento manda**. Los **metadatos**, sí.

PRÁCTICA 5 · 12_buscar.py --explicar

EBIS Business Techschool

PRÁCTICA 6 · BLOQUE I

Responder con fuentes

Respuestas que citan cada afirmación, verificadas por el código. Y leer la fuente.

▸ Ahora, en la terminal

AL TERMINAR

Las slides siguientes: qué verifica el código y qué no.

PASOS

1 La trampa de las versiones

```
python practica/sesion_2/13_preguntar_documentos.py "¿Hasta qué día
estuvo vigente el Pack 4?"
```

→ 29/03 del aprobado; el 31/03 del borrador, solo como antecedente.

2 Una pregunta que da algo por hecho

```
python practica/sesion_2/13_preguntar_documentos.py "¿Cuándo empezaron
las obras?"
```

→ ¿Dice «empezaron» o «programado»? Léelo contra la fuente.

3 En casa (entrega final, ejercicio 1)

```
python practica/sesion_2/14_evaluar_rag.py -k 3
```

→ El examen de recuperación: 4 modelos × 3 búsquedas, sin LLM (~20 s).

Una respuesta con fuentes: cada afirmación, con su cita

*«El Pack 4 estuvo vigente desde el 02/03/2026 hasta el 29/03/2026, ambos incluidos **[F3]**. A partir del 30/03/2026 se retiró el regalo **[F3]**. Esta vigencia fue aprobada y comunicada el 01/03/2026 **[F3]**. La propuesta inicial que indicaba hasta el 31/03/2026 fue sustituida **[F1]**.»*

Cita	Documento	Dónde	Estado	Fecha
[F3]	Calendario comercial aprobado	expediente · pág. 2	aprobado	01/03/2026
[F1]	Propuesta inicial de Pack 4	expediente · pág. 7	sustituido	27/02/2026

✓ **Verificación del código: todas las frases citan, las citas existen y cada fecha aparece en su fuente.**

El código revisa al modelo

Las citas existen
Si cita [F9] y solo hay 6 fragmentos, avisa.

Cada frase lleva cita
«Sin cita, no se afirma» (salvo «no consta»).

Cada cifra está en su fuente
Si dice «30/03/2026» y [F1] no lo contiene, avisa.

Un borrador no es lo vigente
Si cita un sustituido sin decir que es borrador, avisa.

Es el mismo verificador de los datos (sesión 2, bloque I), ahora para **fuentes**: una cifra sin respaldo, **avisa**.

Lo que no puede comprobar: si la frase dice lo mismo que la fuente. Siguiente slide.

Lo que la verificación no ve: la cita es real, el significado no

«¿CUÁNDO EMPEZARON LAS OBRAS?»

«…el inicio efectivo fue el 1 de abril [F2]»

La fuente dice «inicio programado» y que no hay confirmación de la ejecución. La pregunta lo daba por hecho.

«¿QUIÉN LLAMÓ A LA TIENDA Y PARA QUÉ?»

«…se realizó el mismo día que el aviso de obras fue recibido [F1][F5]»

El aviso se recibió el 25/03 y la llamada es del 07/04. Inventa una relación entre fuentes, con cita.

Como el «8 SKU» y las «307 líneas»: comprobar que una cita existe no basta; hay que **leerla**.

Sin consultar, se lo inventa todo… y el código lo para

LO QUE ESCRIBIÓ, SIN CONSULTAR NADA

«…las ventas netas bajaron un 23 % [S1]… los abonos aumentaron un 45 % [S3]. El 28 de marzo se registró un error en el sistema de precios que afectó a 12 productos… [D1]»

Ni el error de precios ni esas cifras existen. Cita [S1] y [D1]… sin haber hecho ninguna consulta.

LO QUE HIZO EL CÓDIGO

0 fuentes

en la lista: las citas no corresponden a nada consultado.

6 cifras sin respaldo

12, 15, 23, 28, 31, 45: no salen de ningún resultado.

Ahora no acepta una respuesta sin consultar antes.

Sin herramientas, el modelo **escribe como un analista**… de memoria. Por eso las fuentes las pone **el código**.

PRÁCTICA SESIÓN 2 · 15_analista.py (respuesta guardada, 11/10)

Test de la sesión 2: las cinco preguntas del RAG (11–15)

#	La idea	Resp.
11	La cabecera (título, código, estado, fecha) se indexa para que el fragmento se entienda solo.	A
12	El borrador lo marca una regla del código («sustituye a…»), no los embeddings ni el LLM.	C
13	BM25 clava códigos; la semántica, sinónimos. La híbrida (RRF) combina posiciones.	A
14	Cita real, significado falso: la búsqueda acertó, falló la redacción. Hay que leer la fuente.	B
15	Sin consultar, cifras inventadas: lo cazó el código. Ahora no acepta respuestas sin consultar.	D

La 12 y la 14 son las dos ideas clave del bloque: las versiones las decide el código, y hay que leer la fuente.

PARA LLEVAR · BLOQUE I

Tres ideas para recordar

1 **Trocear y etiquetar es trabajo del código.**
La ficha (documento, estado, fecha, página) viaja con cada fragmento.

2 **Lo más parecido no es lo vigente.**
Las versiones las resuelve una regla del código, no los embeddings.

3 **Citar no es demostrar.**
El código comprueba que las citas y las cifras existen; el significado, una persona.

Siguiente: lo que no es texto. Un ticket, un PDF escaneado y un gráfico.

EBIS Business Techschool

BLOQUE II · LEER LO QUE NO ES TEXTO

Leer lo que no es texto

Tickets, escaneados, tablas y gráficos: los lee un modelo; lo que dicen, lo comprueba el código.

Pregunta guía
¿Qué se ha leído bien… y cómo lo sabemos?

EN ESTE BLOQUE

1 **Un ticket**
Un VLM lee, un esquema fija los campos y el código compara con caja.

2 **Un segundo encargo**
El cierre de junio: dos PDF que el RAG de la sesión 2 no sabe leer.

3 **Cuatro formas de leer**
Texto del PDF, OCR, parser de documentos y VLM, medidas en el Jetson.

4 **Lector frente a código**
Qué hace PaddleOCR solo y qué programamos: ficha, versión y «sin revisar».

Leer una imagen no es una sola cosa: son cuatro tareas

OCR

RECONOCER TEXTO

```
«TOTAL 231,99 EUR»
```

Caracteres, sin saber qué significan.

→

VLM

INTERPRETAR Y RESPONDER

```
«Es un justificante de 2 artículos por 231,99 €»
```

Entiende la imagen… y puede inventar con aplomo.

→

Extracción

DEVOLVER CAMPOS DEFINIDOS

```
{"total_impreso": 231.99, …}
```

Campos con tipo: el código ya puede usarlos.

→

Verificación

CONTRASTAR CON OTRAS FUENTES

```
caja: 211,99 → no coincide
```

La hace el código, con los registros.

El modelo **lee y propone**; el código **comprueba**. Un VLM no es un notario.

Para un ordenador, una imagen es una tabla de números



Píxeles
Cada punto, un número de 0 (negro) a 255 (blanco); en color, tres: rojo, verde y azul. El ticket: ~4,8 millones de números.

Patrones
Una red aprende sola a reconocer bordes → trazos → letras → palabras y objetos.

Tareas
Clasificar (¿qué es?) · detectar (¿dónde? cajas) · segmentar (¿qué píxeles?) · leer texto (OCR).

Hoy: transformers
La imagen se corta en parches y cada parche se trata como una palabra (ViT, 2020).

A la izquierda: el «2» de «TOTAL 231,99», píxel a píxel (a la mitad de resolución).

Por dentro de un VLM: unos ojos, un traductor y un LLM

Imagen
724 × 1024 px
(reducida por el código)

→

Parches
cuadrados de 16 × 16 px
(como «palabras» de imagen)

→

Codificador de visión
transformer de 27 capas:
cada parche mira a todos

→

Proyector
2 × 2 parches → 1 token
= el «idioma» del LLM

→

LLM (36 capas)
tokens de imagen + texto
→ escribe, token a token

LO QUE ESTO EXPLICA (Y VEMOS HOY)

Más píxeles, más tokens
Original 1060 × 1500: ≈ 1.580 tokens. A 1024 px: ≈ 1.080 y la mitad de tiempo, con el mismo resultado.

Ve trozos, no letras
Una etiqueta pequeña o solapada cae en uno o dos tokens: el 4B lee 95 donde pone 96.

Escribe como un LLM
Lo más probable, no lo comprobado: si no lo ve claro, completa… con aplomo.

Un VLM es un LLM al que le han puesto **ojos**: lee y **propone**; no es un notario.

EBIS Business Techschool

PRÁCTICA 1 · BLOQUE II

Leer y comprobar un ticket

Del ticket al dato comprobado: qué hace el modelo y qué hace el código.

▸ Ahora, en la terminal

AL TERMINAR

Las slides siguientes: el esquema, la forma frente al contenido y la comprobación.

PASOS

1 El ticket de abril

```
python practica/sesion_3/01_ver_imagen.py ticket_abril.png --sin-libre
```

→ JSON, Pydantic, SQL y veredicto: se lee bien… y no cuadra: 231,99 frente a 211,99.

2 La forma no es el contenido

```
python practica/sesion_3/02_validar.py
```

→ Un JSON inventado pasa Pydantic; el código lo caza después.

3 Una foto muy borrosa

```
python practica/sesion_3/01_ver_imagen.py ticket_abril_muy_borroso.png
--guardada
```

→ Valores imposibles, JSON válido → LECTURA DUDOSA.

El esquema lo escribimos nosotros; Pydantic comprueba la forma

asistente/vision.py

```python
class LineaTicket(BaseModel):
    sku: str | None          # None = no se lee
    cantidad: int | None
    importe: float | None

class Ticket(BaseModel):
    pedido_id: str | None
    fecha: date | None       # 2026-04-31 no pasa
    lineas: list[LineaTicket]
    total_impreso: float | None
    campos_ilegibles: list[str]
    legibilidad_declarada: Literal["buena", "regular", "mala"]
```

Lo que comprueba
¿Es JSON? ¿Están los campos? ¿Tienen su tipo? ¿La fecha existe?

Lo que NO comprueba
Que el pedido sea ese, que el total sea correcto o que la imagen diga eso.

Lo que permite
null y campos_ilegibles: decir «no lo leo» en vez de inventar.

Pydantic garantiza la **forma**. El **contenido** se comprueba después.

Cómo obliga «format» a escribir JSON (y por qué no basta)

el modelo va escribiendo…

```
{"pedido_id": "B-MAD-MAL-ORD-0005",
 "fecha": "2026-04-04",
 "total_impreso": 
```

SIGUIENTE TOKEN: LO QUE PROPONE EL MODELO

231	211	"	Tot	}
✓ 0,71	✓ 0,12	✗ 0,09	✗ 0,05	✗ 0,03

Lo que garantiza
La forma: llaves, comillas, campos y tipos. Un número donde va un número. Siempre JSON válido.

Lo que NO garantiza
El valor: entre los números posibles, elige el modelo. 231 o 211, ambos «válidos».

Por eso, después
Pydantic comprueba la forma; el código, el contenido.

Ilustrativo: las probabilidades son un ejemplo. En rojo, los tokens que el esquema prohíbe (romperían el JSON).

JSON válido no es contenido correcto: el ticket muy borroso

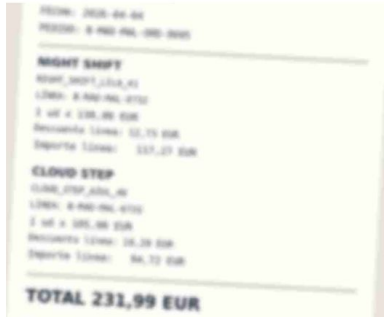
original



borrosa (sintética)



muy borrosa (sintética)



Campo	En la imagen (real)	Lo que leyó
pedido_id	B-MAD-MAL-ORD-0005	8-MAD-MAL-000-0005
fecha	2026-04-04	2020-04-04
línea 1 · importe	117,27	126,13
línea 2 · SKU	…_AZUL_40	…_AZUL_41
legibilidad	—	«buena»

Pydantic: ✓ válido
Todos los campos están y tienen su tipo.

El código: ✗ valores imposibles
Formato de pedido y de línea imposibles, fecha fuera de los datos, el ticket no cuadra consigo mismo.

Veredicto: LECTURA DUDOSA
Repetir la foto o revisar a mano. Y el modelo dijo «buena»: su opinión no es una probabilidad.

PRÁCTICA 1 · 02_validar.py

Comprobar con código: el ticket de abril, campo a campo

Comprobación	En el ticket	Registro / cálculo	
línea 1: cantidad × precio − descuento	117,27 €	117,27 €	✓
línea 2: cantidad × precio − descuento	94,72 €	94,72 €	✓
total impreso = suma de sus líneas	231,99 €	211,99 €	✗
pedido	B-MAD-MAL-ORD-0005	B-MAD-MAL-ORD-0005	✓
fecha	2026-04-04	2026-04-04	✓
tienda	MAD-MAL	MAD-MAL	✓
total: impreso frente a caja	231,99 €	211,99 €	✗

VEREDICTO

DISCREPANCIA

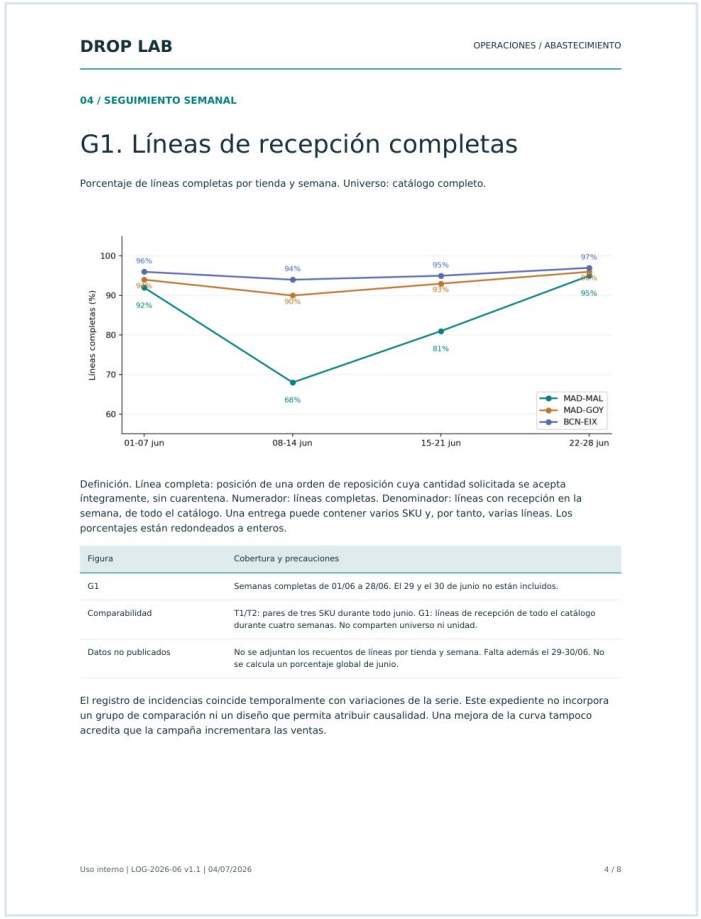
20,00 €

de diferencia en el total

Coincide todo… menos el **total impreso**: 20 € más que sus líneas y que caja. El código lo dice; **no dice por qué**.

PRÁCTICA 1 · 01_ver_imagen.py ticket_abril.png

Segundo encargo: el cierre de junio, en dos PDF



DROP LAB

OPERACIONES / ABASTECIMIENTO

04 / SEGUIMIENTO SEMANAL

G1. Líneas de recepción completas

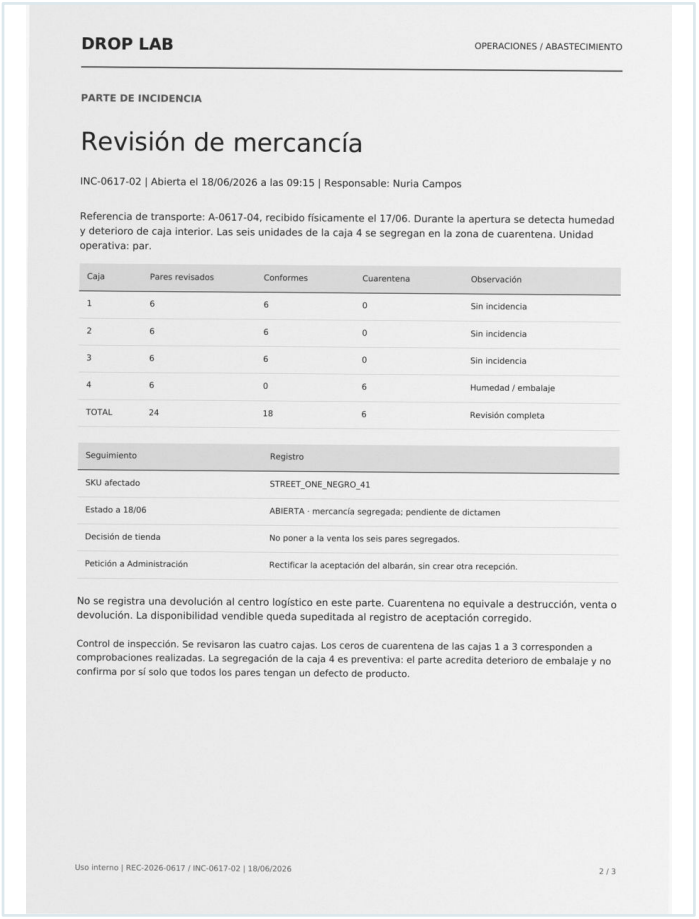
Porcentaje de líneas completas por tienda y semana. Universo: catálogo completo.

Definición. Línea completa: posición de una orden de reposición cuya cantidad solicitada se acepta íntegramente, sin cuarentena. Numerador: líneas completas. Denominador: líneas con recepción en la semana, de todo el catálogo. Una entrega puede contener varios SKU y, por tanto, varias líneas. Los porcentajes están redondeados a enteros.

Figura	Cobertura y precauciones
G1	Semanas completas de 01/06 a 28/06. El 29 y el 30 de junio no están incluidos.
Comparabilidad	T1/T2: pares de tres SKU durante todo junio. G1: líneas de recepción de todo el catálogo durante cuatro semanas. No comparten universo ni unidad.
Datos no publicados	No se adjuntan los recuentos de líneas por tienda y semana. Falta además el 29-30/06. No se calcula un porcentaje global de junio.

El registro de incidencias coincide temporalmente con variaciones de la serie. Este expediente no incorpora un grupo de comparación ni un diseño que permita atribuir causalidad. Una mejora de la curva tampoco acredita que la campaña incrementara las ventas.

Uso interno | LOG-2026-06 v1.1 | 04/07/2026 4 / 8



DROP LAB

OPERACIONES / ABASTECIMIENTO

PARTE DE INCIDENCIA

Revisión de mercancía

INC-0617-02 | Abierta el 18/06/2026 a las 09:15 | Responsable: Nuria Campos

Referencia de transporte: A-0617-04, recibido físicamente el 17/06. Durante la apertura se detecta humedad y deterioro de caja interior. Las seis unidades de la caja 4 se segregan en la zona de cuarentena. Unidad operativa: par.

Caja	Pares revisados	Conformes	Cuarentena	Observación
1	6	6	0	Sin incidencia
2	6	6	0	Sin incidencia
3	6	6	0	Sin incidencia
4	6	0	6	Humedad / embalaje
TOTAL	24	18	6	Revisión completa

Seguimiento	Registro
SKU afectado	STREET_ONE_NEGRO_41
Estado a 18/06	ABIERTA · mercancía segregada; pendiente de dictamen
Decisión de tienda	No poner a la venta los seis pares segregados.
Petición a Administración	Rectificar la aceptación del albarán, sin crear otra recepción.

No se registra una devolución al centro logístico en este parte. Cuarentena no equivale a destrucción, venta o devolución. La disponibilidad vendible queda supeditada al registro de aceptación corregido.

Control de inspección. Se revisaron las cuatro cajas. Los ceros de cuarentena de las cajas 1 a 3 corresponden a comprobaciones realizadas. La segregación de la caja 4 es preventiva: el parte acredita deterioro de embalaje y no confirma por sí solo que todos los pares tengan un defecto de producto.

Uso interno | REC-2026-0617 / INC-0617-02 | 18/06/2026 2 / 3

pág. 4 del informe (gráfico G1) · pág. 2 del expediente (escaneada)

«Necesito el cierre de junio: qué llegó, qué no y cuánto costó.»

Informe de reposición

8 págs. digitales · tablas sin rejilla que saltan de página · un gráfico (imagen) · el borrador v0.9 en un anexo

Expediente escaneado

3 págs. que son solo imagen: albarán, incidencia y rectificación

EBIS Business Techschool

PRÁCTICA 2 · BLOQUE II

Leer los PDF de junio

Separar lo que lee PaddleOCR de lo que programamos y comprobamos nosotros.

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes: el antes, los lectores por dentro, quién hace qué y la ficha.

PASOS

1 Antes: el lector de la sesión 2

```
python practica/sesion_3/08_leer_documentos.py --antes
```

→ Reglas del formato de abril: toma el pie por el código y no puede con el escaneado.

2 Lo que devuelve PaddleOCR, solo

```
python practica/sesion_3/08_leer_documentos.py --capas escaneado 2
--guardada --solo-modelo
```

→ Salida cruda: líneas con su confianza (OCR) y tablas en HTML (PaddleOCR-VL).

3 Lo que hace nuestro código

```
python practica/sesion_3/08_leer_documentos.py --capas informe 8
--guardada --solo-reglas
```

→ Limpia, pone la ficha, une la fila partida a ENT-06-012 y comprueba las cifras.

4 Al índice, con su ficha

```
python practica/sesion_3/09_indice_junio.py
```

→ 62 fragmentos (51 + 11). Con --ver 56: el anexo v0.9, SUSTITUIDO por v1.1.

Antes: el lector de la sesión 2, con los PDF de junio

Qué	Lo que sale	Por qué
Código	Uso interno	Toma la primera línea con dos «|»: aquí es el pie de página.
Estado	sin estado (8 págs.)	El borrador v0.9 del anexo queda igual que el aprobado.
Versiones	«sustituye a LOG-2026-06»	v0.9 y v1.1 tienen el MISMO código: la versión no está en la clave.
Gráfico G1	no existe	Es una imagen: sus 12 porcentajes no están en el texto del PDF.
Tabla T2	fila huérfana en la pág. 8	«tarde. La firma…» es el final de ENT-06-012, en la pág. 7.
Escaneado	IndexError: 0 palabras en la página	No hay texto que extraer: hace falta leer la imagen.

Las reglas de un lector valen para **un formato**. Formato nuevo → reglas nuevas, **y medirlas**.

PRÁCTICA 2 · 08_leer_documentos.py --antes

Cuatro formas de leer una página, medidas en el Jetson

Forma	Herramienta	Escaneado	Informe	Tablas	Gráfico	Confianza
Texto del PDF	pymupdf (CPU)	nada	0,0 s	celda a celda	no	—
OCR clásico	PP-OCRv6 · GPU	1,8 s	2,0 s	no: líneas sueltas	rótulos sueltos	por línea (media 0,995)
Parser de documentos	PaddleOCR-VL-1.6 · GPU	23,8 s	34,1 s	sí	sí: 12/12	no
VLM general	qwen3-vl:8b · Ollama	31,3 s*	33,6 s*	sí	se lo salta	no

*Segundos por página con el modelo cargado (carga: ~10 s el OCR, 5–13 s PaddleOCR-VL). * Una sola página. Sin GPU, el mismo OCR: ~40 s/página.*

Rápido y con confianza **o** con estructura. Y ninguno dice si ha leído **bien**: eso se comprueba.

PRÁCTICA 2 · 08_leer_documentos.py --comparar

Por dentro: lo que PaddleOCR hace solo

OCR clásico · PP-OCRv6
~2 s/pág. en la GPU · no sabe qué es una tabla

Detector
una caja por línea → **Reconocedor**
caracteres, uno a uno → **Salida**
líneas + confianza

Parser · PaddleOCR-VL-1.6
~25–35 s/pág. · el gráfico, como tabla de valores

Maquetación
bloques: título, tabla, gráfico, pie… → **VLM de 0,9 B**
lee cada bloque → **Salida**
Markdown; tablas en HTML

LO QUE DECIDIMOS NOSOTROS ANTES DE LLAMARLA (08_leer_documentos.py)

Página → imagen
150 ppp, con PyMuPDF

En la GPU
motor «transformers» (PyTorch + CUDA)

Gráficos: activados
por defecto, se quedan como imagen

Cabecera y pie: dentro
por defecto los quita; el pie lleva el código

Sale una **lectura**. Código, versión, estado o fila partida: **reglas nuestras**, después.

La misma página: lo que devuelve cada lector de PaddleOCR

PP-OCRv6 · salida cruda: confianza línea

```
1,00  Caja
1,00  Pares revisados
1,00  Conformes
1,00  Cuarentena
1,00  Observación
1,00  1
1,00  6
1,00  6
1,00  0
1,00  Sin incidencia
1,00  2
1,00  6
1,00  6
1,00  0
1,00  Sin incidencia
1,00  3
1,00  6
```

PaddleOCR-VL · su tabla HTML, en filas

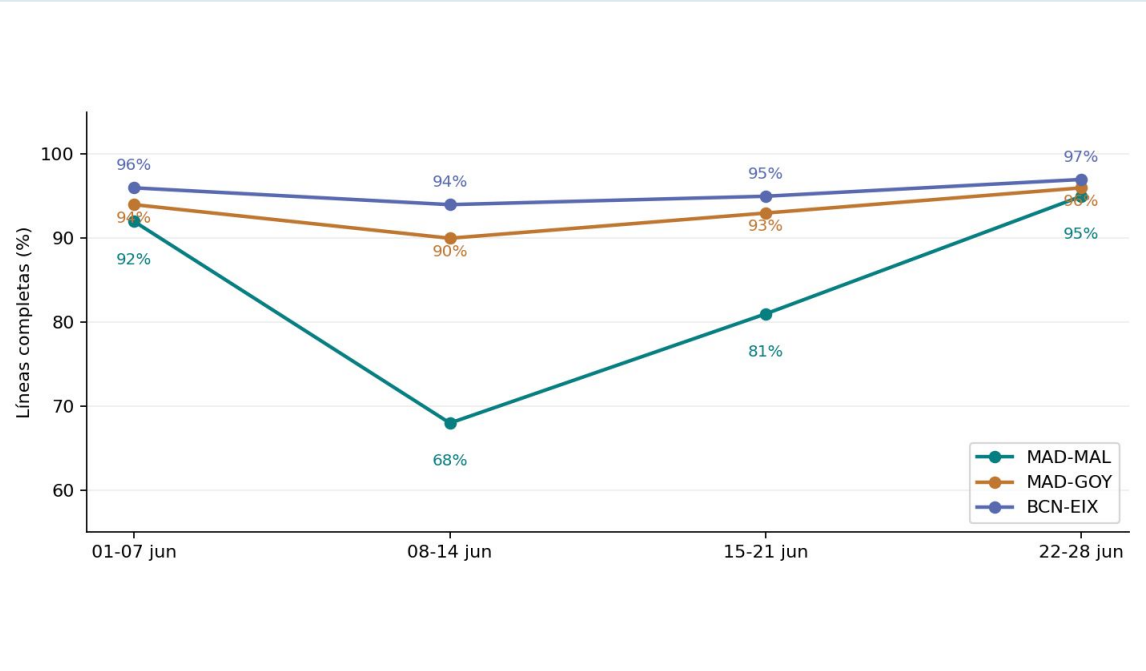
```
| Caja | Pares revisados | Conformes | Cuarentena | Observación |
| 1 | 6 | 6 | 0 | Sin incidencia |
| 2 | 6 | 6 | 0 | Sin incidencia |
| 3 | 6 | 6 | 0 | Sin incidencia |
| 4 | 6 | 0 | 6 | Humedad / embalaje |
| TOTAL | 24 | 18 | 6 | Revisión completa |
```

OCR: líneas sueltas
Cada carácter, bien. Pero ¿qué columna es cada «6»? La tabla se ha perdido.

Parser: una tabla
«Caja 4: 6 revisados, 0 conformes, 6 en cuarentena». Qué documento es, no lo sabe.

PRÁCTICA 2 · 08_leer_documentos.py --capas escaneado 2 --guardada --solo-modelo

El gráfico: sus 12 valores solo están en la imagen



Lector	Qué entra	Aciertos
PaddleOCR-VL-1.6	la página entera	12 / 12
qwen3-vl:8b	el gráfico recortado*	12 / 12
qwen3-vl:4b	el gráfico recortado*	11 / 12
qwen3-vl:8b	la página entera	0 / 12

El 4B lee **95** donde pone 96 (etiquetas solapadas), siempre igual y sin dudar. El 8B con la página entera no lo transcribe: pone un enlace a una imagen que no existe. *Extra: 08_leer_documentos.py --grafico

Leído **no** es comprobado: la ficha marca esas cifras como **SIN COMPROBAR**.

PRÁCTICA 2 · 08_leer_documentos.py --leer informe 4 --con vl --guardada

Qué hace PaddleOCR solo y qué hacemos nosotros

1 · Nosotros, antes
configurar

- Elegir el lector para cada página
- Página → imagen (150 ppp)
- Activar los gráficos y conservar cabecera y pie

`08 · png(), modelo()`

2 · PaddleOCR, solo
la salida cruda

- OCR: líneas, texto y confianza
- Parser: bloques y orden de lectura; lee cada uno
- Tablas en HTML; el gráfico, en tabla
- Página a página: no une páginas

`--solo-modelo`

3 · Nuestro código
después · asistente/lectura.py

- Limpiar: HTML → filas; el pie, aparte
- Reglas: código + versión, estado, «sustituye a», fila partida
- Comprobar cifras con el texto del PDF
- Ficha + «sin revisar»

`--solo-reglas`

4 · Negocio valida
personas responsables del proceso

- ¿«APROBADO» en la pág. 1 vale para todo?
- ¿Esas dos filas son la misma entrada?
- ¿Qué se acepta sin revisar?

antes de usarlo

PaddleOCR **lee**. Limpiar, poner la ficha y comprobar es **nuestro código**; que esas reglas sean ciertas, lo **valida negocio**.

PRÁCTICA 2 · 08_leer_documentos.py --capas escaneado 2 --guardada

Tabla partida y versiones: el modelo lee, la regla es nuestra

ENT-06-011	MAD-GOY STREET_ONE_ARENA_41	20	20	20	Recepción 08/06. Pares aceptados tras comprobar talla y modelo. La fotografía de bultos se conserva en archivo de ruta.
ENT-06-012	MAD-GOY STREET_ONE_ARENA_41	20	20	20	Recepción física registrada el 30/06 a las 18:20 y contrastada con la hoja de descarga del turno de

Uso interno | LOG-2026-06 v1.1 / T2 | 04/07/2026 7 / 8

salto de página (pág. 7 → pág. 8)

DROP LAB OPERACIONES / ABASTECIMIENTO

T2 · Detalle documental	Destino y referencia	Pares de junio			Control de recepción
Registro	Tienda / SKU	Sol.	Exp.	Acept.[1]	Observaciones de conciliación
					tarde. La firma se incorporó el 02/07; el periodo se mantiene en junio por la fecha de recepción y no por la validación administrativa.
ENT-06-013	BCN-EIX STREET_ONE_NEGRO_41	35	35	35	Recepción 02/06. Embalajes exteriores conformes. Cotejo de referencias completado antes del cierre de turno.

Nota (1) del propio informe: «Las casillas vacías en una continuación pertenecen a la misma observación iniciada en la página anterior».

SALIDA CRUDA · PaddleOCR-VL, página a página

```
# pág. 7
| ENT-06-012 | MAD-GOY … | 20 | 20 | 20 | …de descarga del turno de |
# pág. 8
|  |  |  |  |  | tarde. La firma se incorporó… |
```

REGLA NUESTRA · asistente/lectura.py

Una fila **sin registro** al principio de una página es la cola de la **última fila** de la anterior: se pega ahí.

```
| ENT-06-012 | MAD-GOY … | 20 | 20 | 20 | …turno de tarde. La firma… |
```

```
# versiones: la versión entra en la clave
LOG-2026-06 v1.1   aprobado     págs. 1-5, 7-8
LOG-2026-06 v0.9   SUSTITUIDO   pág. 6 (anexo)
# pág. 1: «sustituye a LOG-2026-06 v0.9»
```

PRÁCTICA 2 · 08_leer_documentos.py --capas informe 8 --guardada --solo-reglas

Después: cada página, con su ficha y cómo se leyó

id	Documento · pág.	Código	Estado	Lectura (PaddleOCR-VL, sin revisar)
51	informe · 1	LOG-2026-06 v1.1	aprobado	13/13 valores = texto del PDF
52	informe · 2	LOG-2026-06 v1.1	aprobado	20/20 valores = texto del PDF
53	informe · 3	LOG-2026-06 v1.1	aprobado	21/21 valores = texto del PDF
54	informe · 4	LOG-2026-06 v1.1	aprobado	14 valores solo en la imagen
55	informe · 5	LOG-2026-06 v1.1	aprobado	9/9 valores = texto del PDF
56	informe · 6	LOG-2026-06 v0.9	sustituido → v1.1	18/18 valores = texto del PDF
57	informe · 7	LOG-2026-06 v1.1	aprobado	37/37 valores = texto del PDF
58	informe · 8	LOG-2026-06 v1.1	aprobado	29/29 valores = texto del PDF
59	escaneado · 1	A-0617-04	registro	escaneada: SIN COMPROBAR
60	escaneado · 2	INC-0617-02	abierta (pendiente)	escaneada: SIN COMPROBAR
61	escaneado · 3	RC-0620-01	aprobado	escaneada: SIN COMPROBAR

El índice de la sesión 2 no cambia (51 fragmentos, mismos id). El asistente usa el ampliado: 62.

PRÁCTICA 2 · 09_indice_junio.py --ver 56

PARA LLEVAR · BLOQUE II

Tres ideas para recordar

1 **Un modelo lee; el código comprueba.**
Ticket frente a caja; cifras leídas frente al texto del PDF.

2 **PaddleOCR lee; la ficha y las reglas son nuestras.**
Elegimos el lector y lo medimos; código, versión y fila partida los programamos, y negocio los valida.

3 **Lo que no se ha podido comprobar, se dice.**
«Leído por PaddleOCR-VL, sin revisar» viaja en la ficha hasta la respuesta.

Descanso (10 min). Después: todo esto, detrás de una API y una interfaz web.

EBIS Business Techschool

BLOQUE III · DEL SCRIPT AL ASISTENTE

Del script al asistente

Una API, una interfaz web y un flujo que combina datos, documentos e imágenes.

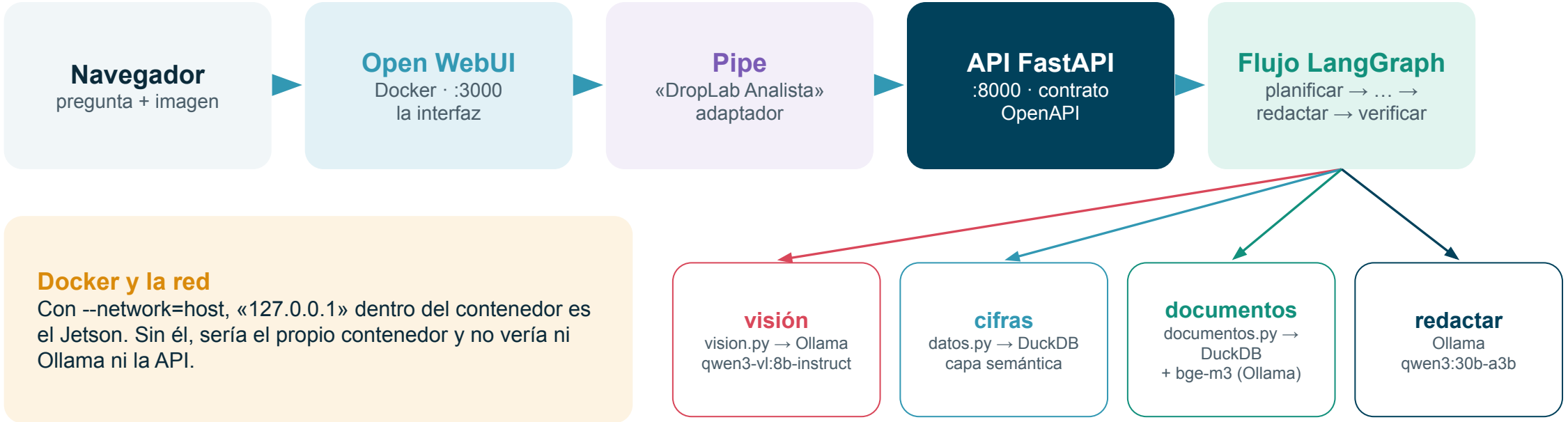
Pregunta guía
¿Quién decide cada paso: el modelo o el código?

EN ESTE BLOQUE

1. **Quién es quién**
   Navegador, Open WebUI, la Pipe, la API y el flujo.
2. **Una API con contrato**
   /docs, seis operaciones y errores que dicen qué pieza falló.
3. **Un flujo con LangGraph**
   Siete nodos: el modelo propone, el código valida y comprueba.
4. **La traza**
   Lo que hizo el sistema, con tiempos: lo que se puede auditar.

La arquitectura: quién es quién

Open WebUI es la **interfaz**; Ollama **ejecuta** modelos; DuckDB **guarda** datos e índice; FastAPI **expone** operaciones; LangGraph **organiza** el proceso.



Docker y la red
Con --network=host, «127.0.0.1» dentro del contenedor es el Jetson. Sin él, sería el propio contenedor y no vería ni Ollama ni la API.

Cinco palabras, con DropLab

Palabra	Qué es	En DropLab
Backend	El programa que procesa las peticiones	La API (FastAPI) y el flujo, en el Jetson
API	La forma acordada de pedirle operaciones	HTTP + JSON en http://<jetson>:8000
Endpoint	Una dirección para una operación concreta	POST /tickets/comprobar · GET /salud
Petición / respuesta	Lo que entra y lo que sale	{pregunta, imagen_base64} → {respuesta, evidencias, errores}
OpenAPI	La descripción del contrato, legible por máquinas	/openapi.json · /docs (con «Try it out»)

Un contrato claro permite cambiar la interfaz (o el modelo) **sin tocar** lo demás.

EBIS Business Techschool

PRÁCTICA 3 · BLOQUE III

La API y el flujo

La misma lógica desde un contrato HTTP y desde un grafo, con su traza.

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes: el grafo, quién decide y la traza.

PASOS

1 La API y su contrato

```
python practica/sesion_3/03_api.py    →    http://<jetson>:8000/docs
```

→ POST /documentos/buscar con una pregunta de junio: la página y su «lectura».

2 Un error que se explica

```
python practica/sesion_3/04_cliente_api.py consulta --pedido "1; DROP
TABLE ventas"
```

→ 422: la API rechaza la entrada antes de llegar al SQL.

3 El grafo

```
python practica/sesion_3/05_flujo.py --dibujar
```

→ Siete nodos y dos decisiones.

4 Una pregunta de junio, de punta a punta

```
python practica/sesion_3/05_flujo.py --caso junio_grafico
```

→ ~15–30 s: plan → documentos → contrastar → redactar → verificar. Con --guardada, sin modelos.

Nuestra API: seis operaciones y un formato de error

Método y dirección	Qué hace	¿Modelo?	Tiempo
GET /salud	Estado de Ollama, índice, datos y modelos	no	< 1 s
POST /documentos/buscar	Fragmentos del RAG (sesión 2 + junio)	solo embeddings	< 1 s
POST /datos/consulta	Una consulta fija con parámetros validados	no	< 1 s
POST /tickets/comprobar	Imagen (multipart) → extraer, validar, consultar, comparar	VLM	~11–25 s
POST /analizar	Pregunta (+ imagen) → el flujo completo	VLM + texto	~8–30 s
POST /analizar/stream	Igual, enviando cada paso al terminar	VLM + texto	igual

```
{"errores": [{"codigo": "PARAMETRO_NO_VALIDO", "componente": "datos", "mensaje": "Formato de pedido no válido: '1; DROP
TABLE VENTAS'"}]}   ← HTTP 422
```

PRÁCTICA 3 · 03_api.py → /docs

LangChain, LangGraph y tres palabras

LangChain

Componentes e integraciones: clientes de modelos, cargadores, divisores, recuperadores.

Hoy no: ya tenemos los nuestros (sesión 2).

LangGraph

Organiza un proceso: pasos, transiciones y estado.

Hoy sí: el flujo del asistente.

Nodo

Una función del proceso.

```
leer_imagen(estado) → lo que cambia
```

Estado

Lo que el proceso conserva entre nodos.

```
pregunta, imagen, consultas,
fragmentos…
```

Enrutamiento

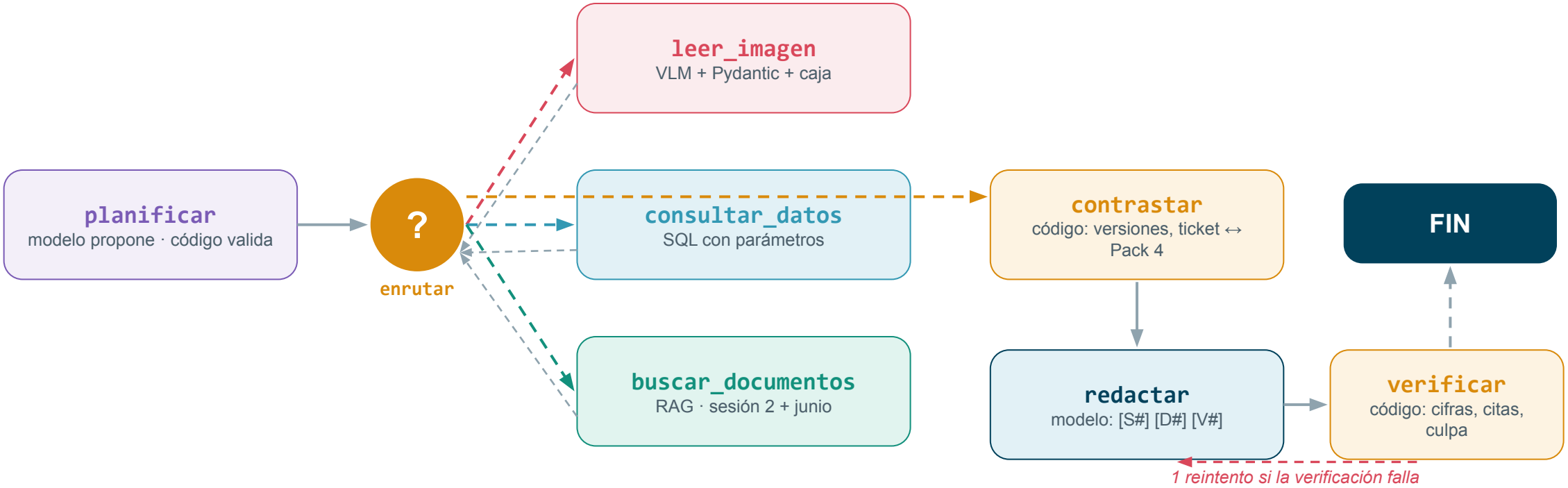
Elegir el siguiente paso.

```
enrutar(estado) → «consultar_datos»
```

Un grafo con estado es **un diagrama que se ejecuta**: se puede dibujar, seguir y medir.

Nuestro grafo: siete nodos y dos decisiones

Continuas: siempre. **Discontinuas: las elige una función de enrutamiento.** Cada tarea vuelve a «enrutar».



Morado: decide el modelo (validado). Amarillo: decide el código. Rojo, azul y verde: herramientas. Azul oscuro: escribe el modelo.

PRÁCTICA 3 · 05_flujo.py --dibujar

Quién decide qué: el modelo propone, el código dispone

Paso	Modelo	Código
Planificar	Propone tareas, tienda y tipo de consulta (JSON con esquema)	Valida con Pydantic; corrige (imagen, pedido, junio → sin SQL); si falla, reglas
Leer la imagen	VLM: extrae campos con esquema	Valida, comprueba formato y cuentas, consulta caja, decide el veredicto
Cifras	— (o SQL del modelo, solo SELECT y sobre la capa semántica)	Consultas fijas con parámetros; las diferencias, en SQL
Documentos	bge-m3: la pregunta → vector	Búsqueda híbrida; versiones (S2) y versión en la clave (junio)
Contrastar	—	Versiones, fuentes automáticas (Whisper, PaddleOCR-VL), ticket ↔ Pack 4, límites
Redactar	Escribe con [S#] [D#] [V#]	Numera las evidencias y fija los apartados
Verificar	—	Citas, cifras, versiones, culpa y causa; 1 reintento

Libertad para el modelo donde aporta (**entender, leer, redactar**); **garantías** del código en el resto.

Contrastar: lo que el código comprueba entre fuentes

Versiones

documentos

expediente pág. 7 SUSTITUIDO por CAM-2026-03-FINAL: solo antecedente.

Fuentes automáticas

documentos

La llamada es Whisper: testimonio. Los PDF de junio, PaddleOCR-VL: «sin revisar».

Ticket ↔ Pack 4

imagen + documentos

04/04 · 2 unidades → no elegible. 16/03 · 4 unidades → elegible (el regalo, no consta).

Límites

la pregunta

Coincidir en el tiempo no prueba una causa. Faltan visitas, regalos entregados, demanda.

El modelo recibe las comprobaciones **ya hechas**: las repite, **no las decide**.

La traza: lo que hizo el sistema, con tiempos

Nodo	Herramienta	Detalle	s
planificar	plan por modelo	imagen → cifras → documentos	1,1
leer_imagen	VLM + Pydantic + SQL	ticket · qwen3-vl:8b-instruct · JSON válido · discrepancia · lectura reutilizada	0,1
consultar_datos	DuckDB · capa semántica	resumen_tienda(MAD-MAL)	0,0
buscar_documentos	RAG · bge-m3 · índice ampliado	1 búsqueda(s) → 5 fragmentos	0,0
contrastar	código	3 comprobación(es) · 2 límite(s)	0,0
redactar	qwen3:30b-a3b	3759 tok. entrada · 509 salida · 33.5 tok/s	19,7
verificar	código	limpia	0,0

21 s en total · texto qwen3:30b-a3b · visión qwen3-vl:8b-instruct

PRÁCTICA 3 · 05_flujo.py --caso combinada --guardada

PARA LLEVAR · BLOQUE III

Tres ideas para recordar

1 **La lógica no se mueve; la API y la interfaz traducen.**
El mismo código en la terminal, en /docs y en Open WebUI.

2 **El modelo propone; el código valida y comprueba.**
Plan con esquema, rutas fijas y contrastes antes de redactar.

3 **La traza es lo que se puede auditar.**
Pasos, herramientas, fuentes y tiempos; no el «razonamiento» del modelo.

Siguiente: el asistente en directo, con siete preguntas difíciles.

EBIS Business Techschool

BLOQUE IV · EL ASISTENTE, EN DIRECTO

El asistente, en directo

Preguntas difíciles de dirección, de punta a punta, en Open WebUI. Las leemos juntos.

Pregunta guía
¿Firmaríais esta respuesta?

EN ESTE BLOQUE

1 **Siete preguntas con trampa**
Versiones, un ticket, una causa, un gráfico, un escaneado y lo que no consta.

2 **Antes de lanzarla**
Entre todos: ¿qué debería contestar y dónde está?

3 **Leer la respuesta**
Cuatro preguntas: correcta, con fuentes, con límites y con traza.

4 **Y después**
Qué mejoraríamos para usarlo de verdad, y lo que nos llevamos.

Siete preguntas difíciles, y dónde está la trampa

#	Tipo	Pregunta	La trampa
1	Versiones	¿Hasta qué día estuvo vigente el Pack 4 en Malasaña?	Lo más parecido es el borrador: decía 31/03
2	Un ticket	¿Qué demuestra este ticket? Compáralo con caja. (+ ticket_abril.png)	Se lee perfecto… y no cuadra con caja
3	Una causa	¿La rotura de stock del 6 de abril es la causa de la caída de Malasaña?	Pide una causa que los datos no prueban
4	Junio · gráfico	¿Qué semana de junio tuvo el peor porcentaje de líneas completas en Malasaña? ¿Fue por la caja húmeda del albarán A-0617-04?	El dato está en una imagen, y la caja llegó después
5	Junio · escaneado	¿Cuántos pares del albarán A-0617-04 se aceptaron al final y qué pasó con el resto?	Solo imagen: nadie lo ha leído a mano
6	Lo que no consta	¿Cuántas tote bags se entregaron en marzo y cuántas ventas se perdieron por las obras?	Invita a inventar dos cifras
7	Una errata	¿La campaña de junio aumentó las ventas de Street One?	En junio no hay ninguna campaña

Las de junio solo están en los PDF de junio: los datos de ventas acaban el 26/04.

EBIS Business Techschool

EN DIRECTO · BLOQUE IV

El asistente, en directo

Saber qué debería contestar antes de preguntar, y comprobar lo que contesta.

▸ Ahora, en Open WebUI

AL TERMINAR
Lo que comprueba el código y lo que tenéis que mirar vosotros.

PASOS

1 Leemos la pregunta y buscamos la trampa

```
Entre todos, 30 segundos
```

→ ¿Qué debería contestar? ¿En qué documento, consulta o imagen estará?

2 La lanzo en Open WebUI

```
«DropLab Analista» · en el proyector (8–30 s)
```

→ Mientras piensa, aparecen los pasos: planificar, buscar, contrastar, redactar, verificar.

3 Abrimos lo que hay debajo

```
Comprobaciones · Fuentes · Traza
```

→ Abrimos la fuente citada y la leemos: ¿dice eso?

4 Cuatro preguntas

```
¿Correcta? ¿Con fuentes? ¿Con límites? ¿Con traza?
```

→ Si las cuatro son «sí», la firmaríamos.

Cómo leer una respuesta: cuatro preguntas

¿Es correcta?

¿Las cifras y fechas son las de la fuente vigente, no las del borrador?

68 %, semana 08–14

¿La respaldan las fuentes?

Abrid la cita: ¿dice exactamente eso? ¿Viene a cuento?

[D2] = informe, pág. 4

¿Dice lo que no sabe?

¿Separa hecho de hipótesis? ¿Evita causas y culpas sin pruebas?

«coincidir no prueba la causa»

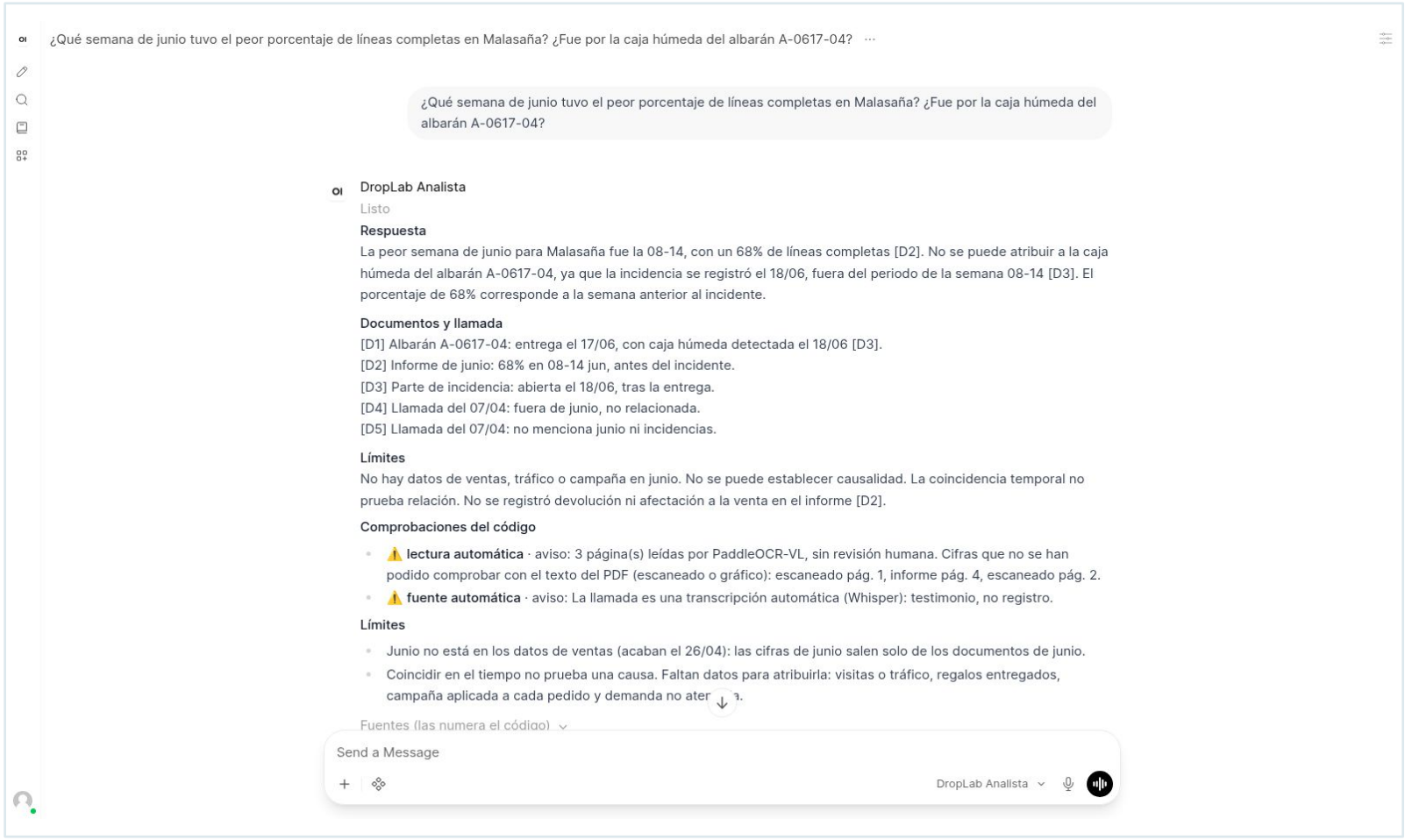
¿Se ve qué hizo?

¿Qué consultó, qué leyó y cómo («sin revisar»)? ¿Cuánto tardó?

«PaddleOCR-VL, sin revisar»

Si las cuatro son **sí**, la firmaríamos. Si alguna es **no**, se dice antes de mandarla.

Así se ve una pregunta de junio en Open WebUI



Respuesta
la redacta el modelo, con [D#]

Comprobaciones del código
lectura automática, versiones…

Límites
junio no está en los datos; causa ≠ coincidencia

Fuentes
desplegable: página, estado y lectura

Traza
pasos, herramientas y segundos

Lo que comprueba el código y lo que miráis vosotros

LO COMPRUEBA EL CÓDIGO, SIEMPRE

- ✓ Que cada cita [S#] [D#] [V#] existe
- ✓ Que cada cifra está en alguna evidencia
- ✓ Que un borrador no pasa por vigente
- ✓ Que no afirma culpas ni causas sin decirlo
- ✓ Que avisa de lo leído por OCR o por Whisper

LO TENÉIS QUE MIRAR VOSOTROS

- ? Que la cita dice exactamente eso
- ? Que la cita viene a cuento
- ? Que la página con el dato llegó a la búsqueda
- ? Que la cifra es de la fila correcta
- ? Que la conclusión tiene sentido para el negocio

«Verificación limpia» = ha pasado **nuestras reglas**. Que sea **verdad**, lo dice quien **abre la fuente**.

CIERRE

Hay mucho más preparado: probadlo

Cómo se conecta todo	guia_integracion.html: la API, el flujo y Open WebUI, archivo a archivo y en orden.
Vuestro cliente	04_cliente_api.py y /docs: cualquier programa puede usar el asistente por HTTP.
Sin framework	05_flujo.py --sin-framework: el mismo proceso con un bucle de Python.
Evaluar por piezas	06_evaluar.py: lectura, recuperación, cálculo, respuesta e integración, por separado.
Elegir configuración	07_comparar.py: 30B + VL 8B frente a 4B + VL 4B, mismas tareas.

Todo admite --guardada (salidas reales, sin modelos). Cada script explica su uso al principio y con --help.

CIERRE

Para usarlo de verdad: lo que mejoraríamos

Calidad medida
Un banco de preguntas reales con respuesta revisada; medir cada cambio contra él.

Verificar mejor
La cifra, en la fuente citada; y una persona revisa lo que dice «sin revisar».

Documentos al día
Entrada automática, versiones desde el gestor documental, tablas partidas resueltas.

Seguridad
Usuarios y permisos; cada uno ve solo sus documentos; registro de quién preguntó qué.

Operación
Trazas y tiempos guardados, alertas si algo cae, cola de peticiones.

Despliegue
Contenedores con versiones fijas, configuración fuera del código y pruebas automáticas.

Las piezas ya están. Llevarlo a producción es **medir**, **proteger** y **operar**.

PARA LLEVAR · LAS TRES SESIONES

Lo que nos llevamos

1 **El código calcula; el modelo lee, entiende y redacta.**
Sesión 1: la regla de oro. Sigue igual con voz, documentos e imágenes.

2 **El negocio, los metadatos y las comprobaciones, en el código.**
Capa semántica, fichas con versión, verificadores y contrastes.

3 **Leer no es comprobar; citar no es demostrar.**
Lo que no se ha podido comprobar, se dice: «sin revisar», «no consta».

4 **Todo en local: los datos no salen de DropLab.**
Un Jetson, modelos de pesos abiertos, PaddleOCR y una interfaz web.

Para seguir: guia_integracion.html · Entrega final: Deberes_sesion_3.docx (individual).

EBIS Business Techschool

EXTRA + · SI SOBRA TIEMPO

Si sobra tiempo

Cinco demos cortas, medidas y con salida guardada. Se pueden lanzar en cualquier orden.

▸ Ahora, en la terminal

AL TERMINAR

Y una quinta: Ctrl+C en la API y preguntar en Open WebUI → «API_NO_DISPONIBLE (pipe)».

PASOS

1 El gráfico, a un VLM pequeño (~7 s)

```
python practica/sesion_3/08_leer_documentos.py --grafico --modelo
qwen3-vl:4b-instruct
```

→ 11/12: lee 95 donde pone 96, sin dudar. Sin --modelo, el 8B: 12/12.

2 Una respuesta convincente… y falsa

```
python practica/sesion_3/05_flujo.py --caso extra_falla
```

→ Dice 60 pares; son 40. Verificación limpia. ¿Qué pieza ha fallado?

3 ¿Y la campaña de junio?

```
python practica/sesion_3/05_flujo.py --caso extra_campana
```

→ La errata de la pág. 4: en junio no hay campaña. ¿Dice «no consta»?

4 Evaluar por piezas, en 5 s

```
python practica/sesion_3/06_evaluar.py --parte integracion
```

→ 10/10 pruebas sin modelo de texto: dónde falla, no solo si falla.

Una respuesta convincente… y falsa: ¿qué pieza ha fallado?

LO QUE DIJO EL ASISTENTE (VERIFICACIÓN LIMPIA)

«MAD-GOY aceptó 60 pares de STREET_ONE_ARENA_41 en junio. La última entrega llegó el 30/06/2026.»

LO QUE DICEN LOS PDF

40 pares: T1 sigue en la pág. 3 (MAD-GOY · ARENA_41 · 40 / 40 / 40).

Última entrega: ENT-06-012, 30/06 a las 18:20: la fila partida entre la pág. 7 y la 8.

DIAGNÓSTICO, PIEZA A PIEZA

Recuperación · aquí
La pág. 3 no está entre los 8 primeros. Llega la pág. 2.

Redacción · hereda
Coge la fila más parecida que sí tiene: NEGRO_41.

Verificación · no lo ve
El 60 existe en las evidencias (otra fila): limpia.

La tabla partida, otra vez: si la página que tiene el dato **no llega**, el modelo **rellena**.

PRÁCTICA EXTRA · 05_flujo.py --caso extra_falla

Una errata del documento: «la campaña» de junio

Pág. 4 del informe: «… Una mejora de la curva tampoco acredita que **la campaña** incrementara las ventas.» ← en junio no hay campaña

LO QUE RESPONDE EL ASISTENTE (18/10)

«No se puede afirmar que la campaña de junio aumentó las ventas de Street One. No hay datos de ventas de junio en las evidencias. El informe de junio solo registra reposiciones y disponibilidad, no resultados comerciales.»

Los documentos también se equivocan: una fuente es **una evidencia**, no la verdad.

PRÁCTICA EXTRA · 05_flujo.py --caso extra_campana

Medido (12/10, modo 30 W): el asistente, pieza a pieza

Pieza	qwen3 30b + 8b	qwen3 4b + 4b	Tiempo por prueba
Lectura visual	6/6 imágenes	6/6 imágenes	56.2 s
Recuperación	6/6 preguntas	6/6 preguntas	0.1 s
Consulta y cálculo	11/11 cifras	11/11 cifras	0.5 s
Interpretación	7/7 preguntas	6/7 preguntas	57.2 s
Integración	10/10 pruebas	10/10 pruebas	0.3 s

Si una pieza falla con todos los modelos, el problema **no es el modelo**: es el dato, el índice o el código.

PRÁCTICA EXTRA · 06_evaluar.py

Dos configuraciones locales (12/10, modo 30 W)

Medida	A · Jetson (30B + VL 8B)	B · Portátil (4B + VL 4B)
Imágenes reales bien leídas	4/4 · 38/38 campos	4/4 · 38/38 campos
Imagen ilegible (sintética)	detectada · lectura_dudosa	detectada · json no válido
Respuestas correctas · verificación limpia	7/7 · 7/7	6/7 · 7/7
s por imagen · s por pregunta (cargado)	56 s · 57 s	50 s · 46 s
Carga (una vez): texto · visión	19 s · 10 s	6 s · 7 s
Memoria: texto + visión	19.8 + 6.5 GB	4.5 + 4.2 GB

Casi empatan: el código hace lo difícil
Cifras, comparaciones y un plan con opciones cerradas. Con SQL libre (sesión 2): 91 % frente a 67 %.

Leer desempata
B omite 108 → 7 en la combinada y dice que la tienda «abrió después» (siguió abierta). Ningún verificador lo ve.

PRÁCTICA EXTRA · 07_comparar.py